Software Test Plan (STP) - File Compression Tool

Project: File Compression Tool (Basic ZIP Implementation)
Version: 1.1
Authors: Team 14
Date: 27-10-2026
Status: Draft

1. Introduction

Purpose: This document defines the test plan for the File Compression Tool (Basic ZIP Implementation). It covers the test objectives, scope, approach, test environment, test cases, responsibilities, risks, and traceability to the SRS.
Scope: Testing covers file input, Huffman-based compression, compressed-file output, decompression, lossless restoration, compression metrics, error handling, security checks, performance, memory use, and portability. The updated SAD defines the compressed output as a custom Huffman-based format and does not claim compatibility with standard ZIP archives.
References: SRS Version 1.1, SAD Version 1.0, and the Software Engineering Mini-Project Deliverables Part-1 document.
Definitions: CLI (Command Line Interface), SRS (Software Requirements Specification), SAD (Software Architecture and Design Specification), RTM (Requirements Traceability Matrix).

2. Test Items

- Command Line Interface (CLI)
- File I/O Module
- Compression Engine
- Huffman Tree Module
- Decompression Engine
- Compression metrics output
- Error and file handling
- Performance and memory behavior

3. Features to be Tested

Features mapped to SRS requirement IDs:
- REQ-1: Accept a valid file path
- REQ-2: Calculate character frequencies
- REQ-3: Build Huffman tree and generate encoded bitstream
- REQ-4: Create the compressed file in the custom format/extension
- REQ-5: Read compressed file and extract tree metadata

- REQ-6: Decode the compressed bitstream
- REQ-7: Write the decompressed file with the original contents and size
- SRS 2.2 / SAD 4.3: Show original size, compressed size, and compression ratio after compression
- SRS 6.1 / SAD 3.1, 3.6: Check time and memory limits for files up to 100 MB
- SRS 6.2 / 6.3 / SAD 3.9: Check error handling, permissions, and corrupted files
- SRS 2.4 / 6.4: Check portability on supported operating systems

4. Features Not to be Tested

- Standard ZIP archive compatibility
- REST or web APIs
- Database features
- GUI features
- Cloud or network storage
- Encryption and password protection
- Features not listed in the SRS or SAD

5. Test Approach / Strategy

Levels:
- Unit tests (frequency calculation, Huffman tree, encoding/decoding, metadata handling, metric calculation)
- Integration tests (CLI, File I/O, Compression Engine, Huffman Tree Module, Decompression Engine)
- System tests (complete compression and decompression using real files)
- Acceptance tests (final system checked against the SRS requirements)

Types:
- Functional testing
- Negative testing
- Edge-case testing
- Performance testing
- Memory testing
- Portability testing
- Security and safety testing
- Regression testing

Entry Criteria: Build is available, CLI syntax is known, test files are ready, and the test environment is ready.

Exit Criteria: Planned tests are executed or marked as blocked/not run, all requirements have test coverage, no critical defect remains open, and performance results are recorded.

5.1 Security Validation

- Check that the application follows operating-system file permissions and does not bypass them
- Try to decompress a corrupted or incomplete compressed file
- Check that a failed write does not damage an existing file
- Check invalid file paths and file-access failures
- Where possible, test insufficient disk space in a controlled environment

6. Test Environment

Hardware: Standard modern computer; record CPU, RAM, and storage for performance tests.
Software: C/C++ compiler and the project build, using standard C/C++ libraries. Interface is the Command Line Interface (CLI). Windows, Linux, and macOS are the target platforms in the SRS/SAD; record which are tested.
Tools: File-size check, byte comparison, time measurement, and memory measurement tools.
Test Data: Text files, binary files, empty file, single-symbol file, valid compressed file, corrupted compressed file, and files up to 100 MB.

7. Test Schedule

Milestones:
- Test plan and test case design: 06-Oct-2026 (updated 27-Oct-2026)
- Test environment and data setup (GitHub Actions CI on Linux, Windows, macOS): 12-Oct-2026 to 16-Oct-2026
- Test execution: Sprint 1, 19-Oct-2026 to 23-Oct-2026 (functional, REQ-1 to REQ-7); Sprint 2, 26-Oct-2026 to 30-Oct-2026 (non-functional and security)
- Regression and final review: 30-Oct-2026 (automated on every pull request)
- Development freeze and submission: 30-Oct-2026

8. Test Deliverables

- Test Plan (this document)
- Test Cases
- Test Data
- Test Execution Log
- Defect Reports
- Performance and Memory Results
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
Developer & Agile Coordinator	Inchara K Kuppal	REQ-4, REQ-7, output files, sizing and memory management

Lead Developer	Pavan Kumar H K	REQ-3, Huffman tree and encoded bitstreams; performance
Developer & QA Lead	Rachit Upadhyay	REQ-5, REQ-6, metadata extraction, decoding and lossless testing
Project Lead & DevOps Engineer	Pranavi Sistla	REQ-1, REQ-2, CLI input validation, file parsing and portability

10. Risks and Mitigation

Risk	Mitigation
Incorrect Huffman tree or encoding	Use known-frequency files and round-trip testing
Corrupted compressed data	Test invalid and truncated files
Large-file processing too slow	Run 100 MB performance tests and record the result
Memory usage too high	Measure peak memory during large-file tests
Permission or disk-space failure	Run controlled negative tests and check existing files
Platform differences	Run core tests on available supported platforms

11. Assumptions & Dependencies

- C/C++ compiler and standard libraries are available
- The CLI syntax and custom file format are defined for the build under test
- The host OS provides normal file permissions
- Test files and output folders can be created for testing
- The 100 MB performance target is measured on a recorded standard modern processor

12. Suspension & Resumption Criteria

Suspend testing if:

- The build cannot be compiled or started
- A defect causes repeated crashes or data loss
- The test environment is unavailable
- A change to the CLI or file format invalidates the current test cases

Resume testing if:
- A corrected build is available and passes a smoke test
- The environment is available again
- The current CLI syntax and file format are confirmed

13. Test Case Management & Traceability

RTM maps the SRS requirements to the test cases below. Update the status during testing.

Test ID	Req.	Test Case	Procedure	Expected Result	Status
TC-COMP-01	REQ-1	Valid file path	Give a valid file path to the compression command.	File path is accepted.	Not Run
TC-COMP-02	REQ-2	Frequency calculation	Run the frequency routine on a file with known character counts.	Calculated frequencies match the input.	Not Run
TC-COMP-03	REQ-3	Huffman tree and encoding	Build the tree for known frequencies and encode the input.	Tree and encoded data are generated correctly.	Not Run
TC-COMP-04	REQ-4	Compressed output	Compress a valid file and inspect the output file.	Compressed file is created in the project custom format/extension.	Not Run
TC-COMP-05	REQ-3	Single-symbol file	Compress a file containing one repeated character.	Compression completes and the file can be restored correctly.	Not Run
TC-COMP-06	SRS 2.2 / SAD 4.3	Compression metrics	Compress a valid file and check the CLI output.	Original size, compressed size and compression ratio are shown.	Not Run
TC-DEC-01	REQ-5	Metadata extraction	Open a valid compressed file for decompression.	Required metadata is read successfully.	Not Run
TC-DEC-02	REQ-6	Bitstream decoding	Decode a valid compressed file.	Compressed data is decoded correctly.	Not Run
TC-DEC-03	REQ-7	Lossless restoration	Compress and decompress a text file and a binary file; compare the files byte-for-byte.	Restored files match the originals exactly and sizes are the same.	Not Run
TC-ERR-01	REQ-1 / SAD 4.4	Invalid path	Give a nonexistent path.	Error message is shown and the program does not crash.	Not Run
TC-ERR-02	SRS 6.2 / SAD 3.9	Corrupted compressed file	Change or truncate a valid compressed file and decompress it.	Corruption is detected/handled safely.	Not Run

TC-NF-01	SRS 6.1	100 MB performance	Compress and decompress a 100 MB file and record time.	Each operation is under 10 seconds on the test machine.	Not Run
TC-NF-02	SRS 6.1	Memory usage	Measure peak memory during large-file processing.	Memory stays within the stated file-size-plus-tree-overhead target.	Not Run
TC-NF-03	SRS 2.4 / 6.4	Portability	Build and run the basic test on each available target OS.	Build and core behavior work on each tested OS.	Not Run

13.1 Requirements Traceability

Requirement	Test Cases
REQ-1	TC-COMP-01, TC-ERR-01
REQ-2	TC-COMP-02
REQ-3	TC-COMP-03, TC-COMP-05
REQ-4	TC-COMP-04
REQ-5	TC-DEC-01
REQ-6	TC-DEC-02
REQ-7	TC-DEC-03
Compression metrics	TC-COMP-06
Performance	TC-NF-01
Memory	TC-NF-02
Portability	TC-NF-03
Security/error handling	TC-ERR-01, TC-ERR-02

14. Test Metrics & Reporting

Metrics collected:

- Number of tests planned and executed
- Number of tests passed, failed, blocked and not run
- Requirement coverage
- Open and closed defects
- Compression/decompression time
- Original and compressed file sizes and compression ratio
- Peak memory used

- Number of successful byte-for-byte comparisons

Reports:

- Test execution log
- Defect reports
- Final test summary report

15. Approvals

Role	Name	Signature / Date
Team Representative		
Reviewer / Evaluator		
Project Approval		